# SPRING AND SPRING BOOT IN JAVA

Muttineni Srinath

# AGENDA

1. Introduction
2. Inversion of Control (IoC) & Dependency Injection (DI) Spring Framework
3. Bean lifecycle
4. Scopes
5. Configurations
6. XML-based, annotation-based, and Java-based configurations
7. Drawbacks of Spring Framework.
8. Spring boot.
9. Layers of the Spring Boot.
10. Auto wire.
11. Annotations.
12. Rest with spring boot.
13. Spring data JPA.

## 🔴 Problems Before Spring (Traditional Java Web Development)

❌ **Manual Object Management** – Developers had to manually create & manage objects

❌ **Complex Web Handling** – Servlets & JSP required a lot of manual configuration

❌ **Difficult Database Transactions** – Developers had to handle commit/rollback manually

❌ **No Built-in Security** – Authentication & authorization had to be coded from scratch

❌ **Slow & Error-Prone Development** – Too much boilerplate code & tight coupling

## ✅ How Spring Made It Easy

✓ **Spring MVC** – Simplifies web development (No more Servlets & JSP)
✓ **Spring Boot** – Auto-configures everything (No XML, No manual setup)
✓ **Spring Data JPA** – Reduces database code complexity
✓ **Spring Security** – Provides built-in authentication & authorization
✓ **Spring Cloud** – Makes microservices development easy
✓ **Spring Transaction Management** – Auto-handles database transactions

# Spring:

**Spring** :
A broad term referring to the entire Spring ecosystem, which includes various projects for enterprise Java development, such as Spring Framework, Spring Boot, Spring Security, Spring Data, etc.

**Spring Framework :**

The Spring Framework is a comprehensive framework that provides tools for building Java applications, including modules for dependency injection (Spring Core), web development (Spring MVC), database access (Spring JDBC), and security (Spring Security).

**Spring Boot :**
Spring Boot is an extension of the Spring Framework that makes it easier to create stand-alone, production-ready applications. It removes boilerplate configurations and provides embedded servers like Tomcat for quick deployment.

**Java Application :**
A Java program that runs on the Java Virtual Machine (JVM).

**Stand-Alone, Production-Ready Application:**
A fully configured Java application that can run independently without requiring additional setup.

# Spring Frame work:

Spring Framework is a comprehensive and versatile platform for enterprise Java development. It is known for its **Inversion of Control (IoC)** and **Dependency Injection (DI)** capabilities that simplify creating modular and testable applications.

**Inversion of Control (IoC)** is a design principle where the control of object creation and dependency management is shifted from the program to a container or framework, such as Spring.

**Dependency Injection (DI)** is a technique where dependencies (objects) are injected into a class rather than being created within the class itself. This promotes loose coupling and enhances testability.

**The concept of Inversion of Control (IoC) in Spring and demonstrated its implementation using:**

1. XML Configuration
2. Java-Based Configuration
3. Annotation-Based Configuration

## XML Configuration:

Uses XML files to define beans and specify dependencies.

**Example:**

```
<beans xmlns="http://www.springframework.org/schema/beans">
<!-- Define EmployeeDAO Bean -->
<bean id="employeeDAO" class="com.example.EmployeeDAO"/>
<!-- Define EmployeeService Bean with Dependency Injection -->
<bean id="employeeService" class="com.example.EmployeeService">
<property name="employeeDAO" ref="employeeDAO"/>
</bean>
</beans>
```

## Title: Defining Beans in XML

The <bean> tag is used to declare a Spring-managed component.

## It requires two key attributes:

id → Unique identifier for the bean.

class → Fully qualified class name of the Java object.

### <property> Tag:

The <property> tag is used in Spring XML configuration to set properties of a bean using setter-based dependency injection. It is placed inside a <bean> definition and assigns values or references to a bean's properties.

name – Specifies the property name in the Java class.
value – Assigns a literal value (String, Integer, Boolean, etc.) to the property.
ref – References another Spring bean by its id, injecting it as a dependency.

**Example:**

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- Bean Definition for Employee -->
    <bean id="employee" class="Employee">
        <property name="name" value="John Doe"/>
        <property name="age" value="30"/>
    </bean>

    <!-- Bean Definition for Company, Injecting Employee -->
    <bean id="company" class="Company">
        <property name="employee" ref="employee"/>
    </bean>

</beans>
```

## Constructor Injection in Spring XML Configuration:

Constructor Injection in Spring is a technique where dependencies are passed through a constructor at the time of object creation. This is done using the <constructor-arg> tag in XML configuration.

## Example:

```
<bean id="beanId" class="fully.qualified.ClassName">
    <constructor-arg value="literalValue"/>  <!-- Injects a primitive value -->
    <constructor-arg ref="anotherBeanId"/>   <!-- Injects another bean -->
</bean>
```

## Key Points about Constructor Injection

- ✅ Used <constructor-arg> to pass values or bean references.
- ✅ Ensures mandatory dependencies are set at object creation.
- ✅ No need for setter methods, unlike setter injection.

**Maven and Gradle:**

# 1. Maven

Maven is a build automation tool used for Java projects. It simplifies project management by handling compilation, dependency management, testing, and packaging through an XML-based configuration file (pom.xml).

**Use of Maven:**

Dependency Management – Automatically downloads required libraries.
Build Automation – Compiles, tests, and packages Java projects.
Project Standardization – Provides a consistent directory structure.
Integration with CI/CD – Used in Jenkins, GitHub Actions, and other CI/CD tools.

# 2. Gradle

Gradle is a modern build automation tool for Java, Kotlin, and other languages. It is faster and more flexible than Maven, using Groovy or Kotlin DSL instead of XML for configuration.

**Use of Gradle:**

Faster Builds – Uses incremental builds to speed up compilation.
Flexible Configuration – Supports both procedural and declarative configurations.
Dependency Management – Automatically resolves and manages dependencies.
Multi-Project Builds – Easier handling of large, modular projects.

# WHICH ONE SHOULD YOU USE?

| Scenario | Use Maven | Use Gradle |
| --- | --- | --- |
| **Simplicity & Standardization** | ✅ Yes | ❌ No |
| **Enterprise Projects** | ✅ Yes | ✅ Yes |
| **Performance (Faster Builds)** | ❌ No | ✅ Yes |
| **Large, Complex Projects** | ❌ No | ✅ Yes |
| **Android Development** | ❌ No | ✅ Yes (Android Studio uses Gradle) |

# Issues if Maven/Gradle Are Missing:

## 1.Dependency Management Issues

You have to manually download and manage all libraries and their versions.
Risk of using conflicting versions of dependencies.

## 2.Build Process Complexity

Without a build tool, compiling the code, running tests, and packaging the project must be done manually.
Managing different environments (dev, test, prod) becomes harder.

## 3.Managing Different Environments (Dev, Test, Prod)

Projects often use **different configurations** for different environments:
**Example:**
•**Development:** Uses a local database (e.g., MySQL)
•**Testing:** Uses an in-memory database (e.g., H2)
•**Production:** Uses a cloud database (e.g., PostgreSQL)
Without a build tool, switching environments means **manually changing config files**, which is risky.
With Maven/Gradle, you can define profiles and switch environments easily:

## 3.Longer Development Time

Build automation is lost, making repetitive tasks manual and error-prone.

Since everything is **manual**, developers waste time repeating the same steps.
A build tool automates the process, so you **focus on coding** instead of manually managing builds.

## 4.Continuous Integration (CI) Issues

CI/CD pipelines rely on build tools (Maven/Gradle) to automate testing and deployment.
Lack of a build tool makes integration with tools like Jenkins or GitHub Actions difficult.

**How CI/CD Works?**
A CI/CD pipeline **automates** these steps.
For example, when you push your code to GitHub, **GitHub Actions** (a CI/CD tool) can:
**1.Automatically compile the code.**
**2.Automatically run tests.**
**3.Automatically deploy the application.**

**5.No Standard Project Structure Maven and Gradle enforce a standard directory layout.**

Without them, different developers may structure the project differently, leading to inconsistency.

**Creating objects manually instead of using Inversion of Control (IoC) and Dependency Injection (DI) can lead to several problems:**

**1. Tight Coupling**

When you manually create objects, the class depends directly on the concrete implementations of its dependencies. This tight coupling makes it harder to:

2. **Poor Testability Due to Manual Object Creation**

When you manually create objects in your code, it becomes difficult to replace them with **mock objects** (fake versions used for testing). This makes testing harder because your code always depends on real objects, like databases or APIs, instead of using controlled test data.

3. **Code Duplication Due to Manual Object Creation**

When dependencies are manually created in multiple places, the same instantiation logic might be repeated across the application. This duplication can lead to:

•More difficult maintenance (changing the creation logic requires updating it everywhere).

•Higher risk of errors due to inconsistent instantiation.

Problems in the Above Code

❌ Code Duplication → Both UserService and OrderService create a DatabaseService object separately.

❌ Harder Maintenance → If you want to change DatabaseService (e.g., use a different database or add logging), you must update every place it is instantiated.

❌ Inconsistent Behavior Risk → If one developer changes object creation in one place but forgets another, the system may behave inconsistently.

**4. Violation of SOLID Principles:**

**1. Single Responsibility Principle (SRP):** A class manually creating its dependencies is responsible for more than its core functionality.

**Problem with SRP Violation**

If a class manually creates its dependencies, it is responsible for both:

1. **Business Logic** (Core Functionality)
2. **Object Creation / Dependency Management**

This adds extra responsibility that does not belong to it.

**2. Dependency Inversion Principle (DIP):** The class depends on concrete implementations rather than abstractions.

If a class depends on a **concrete implementation** instead of an **abstraction (interface/base class)**, it becomes tightly coupled to that implementation.

**3. Open/Closed Principle (OCP):** Adding a new dependency or implementation often requires modifying existing code.

If we modify existing classes whenever new functionality is added, we risk breaking existing behavior.

# 2.Java-Based Configuration:

Java-based configuration in the Spring Framework is an alternative to XML configuration, where the configuration is done using Java classes and annotations. It utilizes the @Configuration and @Bean annotations to define beans and manage dependency injection within a Spring application.

**Key Features of Java-Based Configuration:**

**Annotation-Driven:** Uses annotations like @Configuration, @Bean, @ComponentScan, and @Import.
**Type-Safety:** Ensures type-checking at compile time, reducing errors.
**Improved Readability:** Code-based configuration is easier to read and maintain compared to XML.
**Integration with Other Spring Features:** Works well with component scanning and auto-wiring.

# Annotations Used in Java-Based Configuration

- @Configuration – Marks a class as a Spring configuration class.
- @Bean – Declares a method that returns a Spring-managed bean.
- @ComponentScan– Specifies base packages for scanning components.

## Advantages of Java-Based Configuration

✅ **No XML Needed** – Eliminates the complexity of large XML configuration files.

✅ **Refactoring-Friendly** – Java classes are easier to refactor and maintain.

✅ **Better IDE Support** – Autocompletion and validation in modern IDEs.

✅ **Encapsulation of Configuration** – Configuration is contained within Java classes.

## Disadvantages

❌ Can lead to **large configuration classes** if not modularized properly.

❌ Slightly more **boilerplate code** compared to XML-based configuration.

## Annotation-Based Configuration:

Annotation-Based Configuration in Spring is an alternative to XML-based configuration, where annotations are used to define beans, dependencies, and configurations. It simplifies development by reducing boilerplate code and making the application more maintainable.

### 1. Component Scanning and Bean Definition

@Component - Marks a class as a Spring-managed component.
@Service - Specialized version of @Component, used for service layer classes.
@Repository - Specialized version of @Component, used for DAO (Data Access Object) classes.
@Controller - Specialized version of @Component, used in Spring MVC for defining controllers.

### 2. Dependency Injection (DI) Annotations

@Autowired - Automatically injects dependencies.
@Qualifier - Specifies which bean to inject when multiple beans of the same type exist.
@Primary - Marks a bean as the default choice when multiple beans of the same type exist.

**3. Configuration and Bean Management:**

@ComponentScan to detect beans automatically.

@Lazy - Specifies that a bean should be lazily initialized.

@Scope - Defines the scope of a bean (singleton, prototype, etc.).

# MODULES IN SPRING BOOT:

The **Spring Framework** is a comprehensive framework for enterprise Java development. It is **modular**, meaning you can use only the modules you need without adding unnecessary dependencies.

- ## What Does This Mean in Practice?

**1.No Forced Dependencies** – If your application only needs **Spring Core** (for Dependency Injection), you don't need to include Spring MVC, Spring Security, or Spring ORM.

**2.Better Performance & Smaller Size** – Avoids bloated applications with unnecessary libraries, reducing memory usage and startup time.

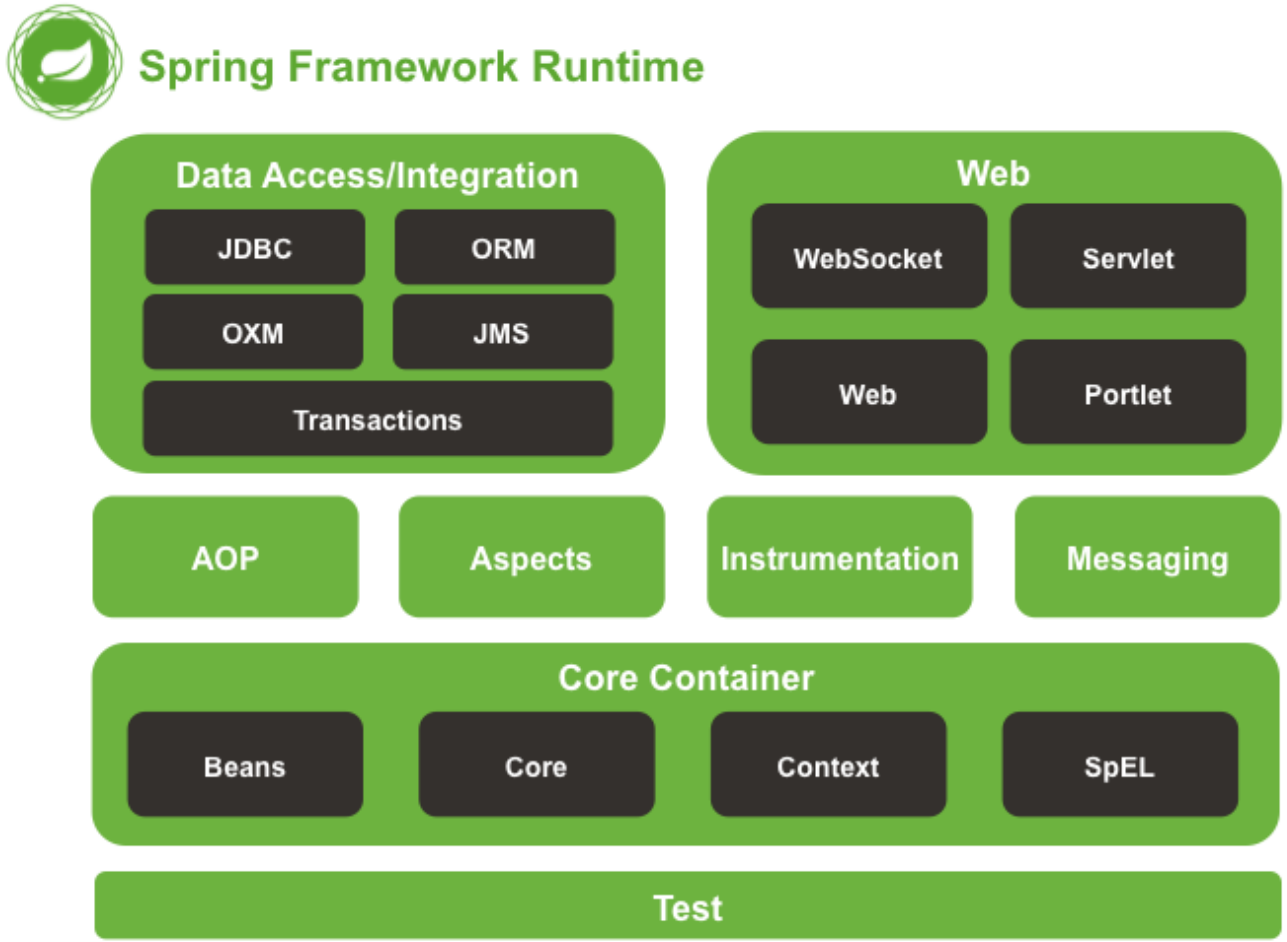
**3.Customizable Configuration** – You can pick and choose specific modules based on your project's needs.

**All the core features provided by Spring (like dependency injection, transaction management, security, ORM, AOP, and web frameworks) were already available in Java EE (J2EE),** but their implementation was complex, required a lot of **manual work**, and had several drawbacks.

How J2EE (Before Spring) Was More Difficult?

| Feature | J2EE (Before Spring) | Spring (After Improvement) |
|---|---|---|
| **Dependency Injection (DI)** | Manual object creation & lookup using **JNDI**. | Automated DI with **IoC Container (@Autowired)**. |
| **Transaction Management** | Required **EJBs, JTA**, and lots of XML configuration. | Simple **@Transactional** annotation. |
| **Database Access (JDBC/ORM)** | Manual **connection handling, transactions, and exception handling** in JDBC. | **Spring JDBC/ORM** simplifies it with minimal code. |
| **Web Framework (MVC)** | **Struts, JSF** were XML-heavy and rigid. | **Spring MVC** is lightweight, flexible, and annotation-based. |
| **Security** | Manually implemented **authentication & authorization**. | **Spring Security** provides built-in authentication & role-based access. |
| **AOP (Aspect-Oriented Programming)** | No built-in support; had to be coded manually. | **Spring AOP** allows easy logging, security, and transaction management. |
| **Configuration** | XML-heavy, **boilerplate code** (deployment descriptors). | **Spring Boot auto-configuration** removes XML clutter. |
| **Server Dependency** | Required a **Java EE Application Server** (GlassFish, JBoss, WebLogic). | **Spring Boot** allows running apps **independently with embedded Tomcat**. |

# MODULES IN SPRING FRAMWORK:



# Test module:

In the **Spring Framework**, the **test module** provides support for **unit testing** and **integration testing** of Spring applications. It is designed to work with **JUnit** and **TestNG** to help developers test Spring components such as controllers, services, repositories, and configurations.

- **Key Features of Spring Test Module:**

**1. Spring Testing Support for JUnit & TestNG**

Supports **JUnit 4, JUnit 5**, and **TestNG** for writing and executing tests.

**2. Testing with Mock Objects**

•Supports **MockMvc** for testing Spring MVC controllers.
•Provides **Mockito** and **Spring Boot Test** utilities for mocking dependencies.

**3. Spring Boot Testing Enhancements**

•Includes Spring Boot Test module (spring-boot-starter-test).
•Supports embedded databases and in-memory databases like H2 for testing.

## 1. Spring Core Container (Core, Beans, Context, Expression Language):

This module is the **heart of Spring**. It provides the basic functionality for dependency injection and bean management.

- **Key Components**

### 1.Spring Core & Beans

1. Provides the **IoC (Inversion of Control)** container.
2. Manages the lifecycle and dependencies of beans (Java objects).
3. Uses **XML or Java-based configuration** to define beans.

### 2.Spring Context

1. Extends the Core module and provides a more powerful application context.
2. Supports internationalization, event propagation, and resource loading.

### 3.Spring Expression Language (SpEL)

1. Used for **manipulating objects dynamically** at runtime. Spring Expression Language (SpEL) is a feature in the Spring Framework that allows you to **dynamically** access and manipulate objects, variables, and methods at runtime.
2. Can be used in XML or annotations.
   Example:

```
@Value("#{10 + 20}")  // Injects 30
 private int sum;
@Value("#{systemProperties['user.name']}")  // Gets current username
 private String username;
```

## Spring AOP (Aspect-Oriented Programming) :

**Aspect-Oriented Programming (AOP)** is a programming paradigm that provides a way to modularize concerns that affect multiple classes (called cross-cutting concerns). Instead of mixing these concerns within business logic, AOP allows defining them separately and applying them declaratively.

**Example of Cross-Cutting Concerns:**

- **Logging**
- **Security (Authentication & Authorization)**
- **Transaction Management**
- **Exception Handling**
- **Performance Monitoring**
- **Caching**

Spring AOP achieves **aspect-oriented programming** by dynamically creating **proxy objects** that intercept method calls and apply additional logic before or after execution.

# Instrumentation:

Instrumentation is the process of **monitoring and measuring** application performance, execution flow, and resource usage. In Spring, instrumentation allows developers to **track application behavior, collect metrics, and diagnose issues** in real-time.

- **Why is Instrumentation Important?**

•Helps in **performance monitoring**.
•Allows **debugging and tracing** execution flow.
•Supports **logging and auditing**.
•Enables **profiling and resource utilization analysis**.

- **How Does Spring Provide Instrumentation?**

•Spring provides various tools and integrations for instrumentation:

•**Spring Actuator** – Provides built-in endpoints for monitoring (e.g., /metrics, /health).
•**Micrometer** – A metrics collection library that integrates with Prometheus, Graphite, New Relic, etc..
•**AOP (Aspect-Oriented Programming)** – Can be used for logging and tracking method execution.
•**JMX (Java Management Extensions)** – Allows managing and monitoring Java applications.
•**Distributed Tracing (Zipkin, Jaeger)** – Helps track requests across microservices.

# Messaging in Spring:

Messaging is a communication mechanism that allows different applications, services, or components to **exchange data asynchronously** using message brokers.

- **Why is Messaging Important?**

•**Decouples services** (no direct dependency between producer & consumer).
•**Improves scalability** (systems can process messages independently).
•**Supports asynchronous communication** (reduces response time).
• **Ensures reliability** (messages are not lost).

- **How Does Spring Provide Messaging?**

Spring provides **Spring Messaging** to integrate with different messaging systems.

- **Messaging Components in Spring**

1. **Spring JMS (Java Message Service)** – Integrates with **ActiveMQ, Artemis, etc.**
2. **Spring Kafka** – Supports **Apache Kafka** messaging.
3. **Spring RabbitMQ (AMQP)** – Supports **RabbitMQ** for distributed messaging.
4. **Spring Cloud Stream** – Helps in **event-driven microservices** using messaging.

- **Data Access and Integration Layer in Spring:**

The **Data Access and Integration Layer** in Spring is responsible for managing database interactions and integrating various data sources. It provides a **consistent, flexible, and scalable** way to work with databases

.

| Technology | Description |
| --- | --- |
| **Spring JDBC** | Low-level JDBC API abstraction with simplified exception handling. |
| **Spring ORM** | Integration with ORM frameworks like Hibernate, JPA, and MyBatis. |
| **Spring Data JPA** | Simplifies working with JPA-based repositories. |
| **Spring Data JDBC** | Lightweight alternative to JPA for relational databases. |
| **Spring Data MongoDB** | Supports NoSQL databases like MongoDB. |
| **Spring Data Redis** | Provides integration with Redis for caching. |

- **Data Integration Layer:**
- The **Data Integration Layer** handles **communication with external systems**, such as APIs, messaging systems, and third-party services.

| Technology | Description |
| --- | --- |
| **Spring REST Template/WebClient** | Call external REST APIs. |
| **Spring Messaging** | Integrate with message brokers like RabbitMQ and Kafka. |
| **Spring Batch** | Process large volumes of data efficiently. |
| **Spring Integration** | Provides messaging, event-driven processing, and file handling. |
| **Spring Cloud Stream** | Simplifies event-driven microservices communication. |

**Spring Web Module** 🌐 :

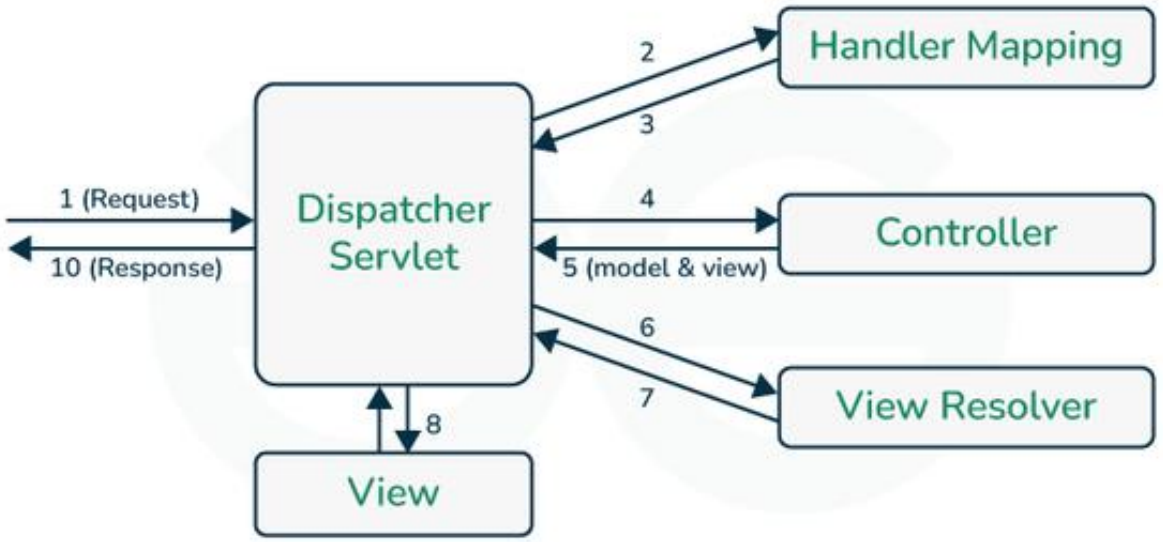
The **Spring Web Module** is part of the **Spring Framework** that enables the development of web applications, including **RESTful APIs, MVC-based web applications, and reactive applications**. It provides powerful features such as **Spring MVC, WebFlux, REST API support, and templating engines**.

- Key Components of the Spring Web Module:

| Component | Description |
| --- | --- |
| **Spring MVC** | Supports **Model-View-Controller (MVC)** architecture for building web applications. |
| **Spring REST (Spring Web)** | Provides features to build **RESTful APIs** using annotations like @RestController. |
| **Spring WebFlux** | A **reactive programming model** for handling non-blocking, asynchronous web applications. |
| **Spring WebSockets** | Supports **real-time bi-directional communication** over WebSockets. |
| **Spring Cloud Gateway** | A **gateway for microservices** that provides routing, load balancing, and security. |

# SPRING MVC BASIC OVERVIEW:

- **Spring MVC (Model-View-Controller) is a framework within the Spring Framework that is used to build web applications following the MVC design pattern. It provides a structured way to develop scalable, flexible, and testable Java web applications.**



# Key Components of Spring MVC

**DispatcherServlet:**

Acts as the front controller that handles all incoming requests.
Dispatches requests to the appropriate handler/controller.

**Controller**

Processes user requests and returns appropriate responses.
Annotated with @Controller or @RestController.

**Model**

Represents the application's business logic or data.
Objects are passed between controllers and views.

**View:**

Responsible for rendering the UI.
Common view technologies: JSP, Thymeleaf, FreeMarker.

**ViewResolver**

Maps logical view names to actual views.
Example: Resolving "home" to "home.jsp".

**HandlerMapping**

Determines which controller method should handle a request.

**Spring MVC Workflow:**

1. The **client** sends a request.
2. The **DispatcherServlet** intercepts the request.
3. The **HandlerMapping** determines the appropriate controller.
4. The **Controller** processes the request and interacts with the **Model**.
5. The **ViewResolver** determines the appropriate view.
6. The **View** is rendered and returned to the client.

**Example of a Simple Spring MVC Controller:**

```
@Controller
@RequestMapping("/hello")
public class HelloController {

    @GetMapping
    public String sayHello(Model model) {
        model.addAttribute("message", "Hello, Spring MVC!");
        return "hello";  // Logical view name (resolved to hello.jsp or hello.html)
    }
}
```

## Flow of Execution:

1. A user accesses http://localhost:8080/home.
2. DispatcherServlet receives the request.
3. HandlerMapping maps "/home" to HomeController.home().
4. HomeController processes the request and adds data to the Model.
5. ViewResolver resolves "home" to home.html.
6. The view is rendered with the message "Welcome to Spring MVC!".

# Problems with Traditional Spring MVC Approach (Without Spring Boot)

**1. Boilerplate Code & Configuration Overhead**

Web.xml for DispatcherServlet Configuration

Developers had to manually configure web.xml to initialize the DispatcherServlet.

**2. No Embedded Server (Need External Tomcat)**

Traditional Spring MVC apps required an external Tomcat server.

Developers had to manually deploy the .war file in Tomcat.

**3. Complex Dependency Management**

•Manually adding dependencies like Spring Core, Spring MVC, Jackson, Hibernate, etc.

•Dependency conflicts were common.

✅ **Spring Boot Solution**: Uses **Spring Boot Starters**, which automatically handle dependencies.

**4. Difficulties in Production Deployment**

Manually configuring WAR packaging.

Need to deploy it on a separate Tomcat/WebSphere server.

✅ Spring Boot Solution:

JAR-based deployment (No need for external servers).

## Spring Boot simplifies Spring MVC development by:

- ✓ Eliminating boilerplate configuration
- ✓ Providing an embedded server
- ✓ Simplifying dependency management
- ✓ Making deployment easier

# Spring boot:

Spring Boot is a framework built on top of the **Spring Framework** that simplifies the process of building and deploying Spring-based applications. It eliminates a lot of boilerplate configuration, making it easier and faster to develop Java applications.

**Key Features of Spring Boot**
**Auto-Configuration:** Automatically configures the application based on dependencies in the classpath.
**Standalone Applications:** No need to deploy to an external web server; Spring Boot comes with embedded servers like Tomcat, Jetty, or Undertow.
**Spring Boot Starter Dependencies:** Simplifies dependency management by providing predefined "starters" (e.g., spring-boot-starter-web for web apps).
**Production-Ready Features:** Includes built-in support for health checks, metrics, logging, and externalized configuration via Spring Boot Actuator.
**Minimal XML Configuration:** Uses convention over configuration and relies on annotations (@SpringBootApplication) instead of XML.

## Inversion of Control (IoC) and Dependency Injection (DI) in Spring Boot :

In **Spring Boot**, the following annotations **replace** the need for manual configuration in the Spring Framework:

# @SpringBootApplication (Replaces Multiple Configurations)

👉 Replaces:

@Configuration (for Java-based configuration)
@EnableAutoConfiguration (for auto-configuring beans)
@ComponentScan (for scanning components)

## 1. Constructor Injection

Injects dependencies via a class constructor.
Recommended for mandatory dependencies because it ensures that dependencies are set at object creation.
Encourages immutability since dependencies are declared as final.

**Example:**

Here, Repository is injected via the constructor when the MyService bean is created.

```
@Component
class MyService {
    private final Repository repo;

    @Autowired
    public MyService(Repository repo) {
        this.repo = repo;
    }
}
```

## 2. Setter Injection

Injects dependencies using setter methods.
Useful for optional dependencies, where you may want to set a default value if no dependency is provided.
Allows modification of dependencies after object creation.
**Example:**

**Here, Repository is injected using a setter method.**

```
@Component
class MyService {
    private Repository repo;

    @Autowired
    public void setRepo(Repository repo) {
        this.repo = repo;
    }
}
```

## 3. Field Injection

Injects dependencies directly into fields using @Autowired.
Simpler to implement but makes unit testing harder because dependencies cannot be easily mocked.
Not recommended for large projects due to reduced flexibility and maintainability.
**Example:**

```
@Component
class MyService {

    @Autowired
    private Repository repo;

}
```

Here, Repository is injected directly into the field.

## 4. Interface-based Injection

Uses an interface to define a contract and inject different implementations dynamically.
Common in Java EE patterns and useful for decoupling dependencies.

**Example:**

```
@Component
class PaymentProcessor {
    private final PaymentService paymentService;

    @Autowired
    public PaymentProcessor(@Qualifier("creditCardService") PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void makePayment(double amount) {
        paymentService.processPayment(amount);
    }
}
```

## Which DI Type Should You Use?

- **Use Constructor Injection** for required dependencies (Best Practice ✅).
- **Use Setter Injection** for optional dependencies.
- **Avoid Field Injection** unless it's a simple case.
- **Use Interface-based Injection** for modular and scalable applications.

**Spring Boot with REST is an evolution of Spring MVC:**

**What is REST?**
**REST (Representational State Transfer)** is an architectural style for designing networked applications. It relies on a stateless, client-server, cacheable communication protocol—typically, HTTP.

- It is based on **stateless communication** between the client and server.
- Uses **HTTP methods** (GET, POST, PUT, DELETE) for operations.
- REST APIs return data in formats like **JSON** or **XML**.

## Principles of REST

1. **Stateless** – Each request contains all necessary information, no session storage.
   - ✓ **Each request is independent** and does not rely on previous requests.
   - ✓ The server **does not store user sessions** between requests.
2. **Client-Server Architecture** – Separation of concerns between frontend and backend.
3. **Cacheable** – Responses can be cached for better performance.

**Uniform Interface** – Standardized way of accessing resources via URIs.

## Why Use Spring Boot for REST APIs?

- ✅ **Built-in Web Server:** Tomcat, Jetty, Undertow.
- ✅ **Minimal Configuration:** Reduces boilerplate code.
- ✅ **Spring MVC Integration:** Simplifies request handling.
- ✅ **Database Integration:** Works seamlessly with JPA, Hibernate.
- ✅ **Security Features:** Supports JWT, OAuth.
- ✅ **Production-ready Features:** Health checks, monitoring, and logging.

## HTTP Methods in REST API:

GET – Retrieves data (e.g., GET /products)
POST – Creates a new resource (e.g., POST /products)
PUT – Updates an existing resource (e.g., PUT /products/{id})
DELETE – Removes a resource (e.g., DELETE /products/{id})
PATCH - Update partial data. (e.g @PatchMapping("/users/{id}"))

# REST API Structure in Spring Boot

**Model (Entity)** → Represents database objects as Java classes with annotations such as @Entity, @Table, and @Id to define database tables and their relationships.

**Repository** → An interface extending JpaRepository or CrudRepository, providing methods for database operations such as saving, deleting, and querying entities.

**Service** → A class annotated with @Service that contains business logic, interacts with the repository layer, and ensures data integrity and processing before reaching the controller.

**Controller** → A class annotated with @RestController that defines REST endpoints using @GetMapping, @PostMapping, @PutMapping, and @DeleteMapping to handle client requests and return responses.

**Spring Boot REST API Annotations:**

@RestController → Defines a REST API controller that handles HTTP requests and returns JSON or XML responses.

@RequestMapping("/api") → Maps HTTP requests to a specific base URL for all methods in the controller.

@GetMapping("/{id}") → Handles HTTP GET requests and retrieves resources based on the provided ID.

@PostMapping → Handles HTTP POST requests to create new resources.

@PutMapping("/{id}") → Handles HTTP PUT requests to update existing resources by ID.

@DeleteMapping("/{id}") → Handles HTTP DELETE requests to remove a resource by ID.

@RequestBody → Converts incoming JSON request data into a Java object for processing.
@ResponseBody  Converts Java objects to JSON automatically

@CrossOriginEnables Cross-Origin Resource Sharing (CORS)

@PathVariable → Captures values from the URL path and binds them to method parameters.
@RequestParam  Extracts query parameters from the URL

**@RequestMapping:**

**Purpose:**
Defines the base URL for all methods inside a controller.

| Attribute | Description | Example |
| --- | --- | --- |
| value / path | Defines the base URL | @RequestMapping("/users") |
| method | Specifies allowed HTTP methods | @RequestMapping(value = "/users", method = RequestMethod.GET) |
| produces | Defines response format | @RequestMapping(value = "/users", produces = "application/json") |
| consumes | Defines accepted request format | @RequestMapping(value = "/users", consumes = "application/json") |

# @CrossOrigin:

**Purpose:**

Enables **CORS (Cross-Origin Resource Sharing)**, allowing requests from different domains.

| Attribute | Description | Example |
| --- | --- | --- |
| origins | Allowed origins | @CrossOrigin(origins = "http://example.com") |
| methods | Allowed HTTP methods | @CrossOrigin(methods = {RequestMethod.GET, RequestMethod.POST}) |
| maxAge | How long results are cached | @CrossOrigin(maxAge = 3600) |

When your frontend (React, Angular, etc.) makes a cross-origin request to a backend API, the browser first sends a preflight request (OPTIONS request) to check if the server allows it.

- Without maxAge → The browser sends a new preflight request every time, increasing network traffic.

- With maxAge → The browser remembers the server's CORS policy for a set time (e.g., 1 hour), so it doesn't need to send preflight requests again during that period.

✓ This improves performance by reducing unnecessary requests. 🚀

# @GETMAPPING,@POSTMAPPING,@POSTMAPPING

| Attribute | Description | Example |
| --- | --- | --- |
| value / path | Defines the endpoint URL | @PutMapping("/users/{id}") |
| consumes | Specifies accepted request type | @PutMapping(value = "/users/{id}", consumes = "application/json") |

# @PATHVARIABLE:

PURPOSE:
EXTRACTS VALUES FROM THE URL PATH AND BINDS THEM TO METHOD PARAMETERS.

| Attribute | Description | Example |
| --- | --- | --- |
| value | Name of the path variable | @PathVariable("id") Long userId |
| required | Makes it optional (default: true) | @PathVariable(required = false) String name |

# @REQUESTPARAM:

**PURPOSE:**

EXTRACTS QUERY PARAMETERS FROM THE URL

| Attribute | Description | Example |
| --- | --- | --- |
| value / name | Query parameter name | @RequestParam(name = "username") String userName |
| defaultValue | Provides a default if missing | @RequestParam(defaultValue = "Guest") |
| required | Makes it optional (default: true) | @RequestParam(required = false) String email |

# Problems Before with try catch exception handling:

**1 Code Duplication**
- Every controller had to handle exceptions separately.
- Same try-catch logic repeated in multiple places.
- Hard to maintain when changes are needed.

**2 Tightly Coupled Code**
- Exception handling was mixed with business logic.
- Reduced code readability and maintainability.
- Harder to debug and update.

**3. Increased Boilerplate Code**
- Manually handling exceptions led to extra lines of code.
- More effort required for writing and maintaining code.

**4 Difficult to Manage Application-Wide Errors**
- No centralized mechanism for handling exceptions.
- Had to modify each controller separately for changes.

# Global exception handler:

Global Exception Handling in Spring Boot allows us to handle exceptions centrally instead of writing repetitive try-catch blocks in each controller. This is done using @ControllerAdvice and @ExceptionHandler.

👉 **With Global Exception Handling:**

•All exceptions are handled in one place.
•Business logic remains clean.
•Consistent and structured error responses.

## @ControllerAdvice & @RestControllerAdvice – What Are They?

•@ControllerAdvice and @RestControllerAdvice are global exception handling mechanisms in Spring Boot.

•They allow us to handle exceptions across multiple controllers in a centralized way instead of writing try-catch blocks in every controller.

# @ControllerAdvice:

It is used to handle exceptions for all controllers in a Spring application.
It works with both REST controllers (@RestController) and normal controllers (@Controller).
Used when you need to handle exceptions and return views (HTML pages) instead of JSON.

**EXAMPLE:**

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public String handleUserNotFoundException(Model model, UserNotFoundException ex) {
        model.addAttribute("error", ex.getMessage());
        return "error-page"; // Returns an error page (Thymeleaf, JSP, etc.)
    }
}
```

**NOTE:** This is useful for **Spring MVC applications** that return **HTML pages**.

# @RestControllerAdvice (Specialized for REST APIs)

It is a combination of @ControllerAdvice + @ResponseBody.
Used for REST APIs where the response is JSON instead of an HTML page.
Automatically converts the exception handling response into JSON format.

**EXAMPLE:**

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFoundException(UserNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }
}
```

**NOTE:** This is used when you are **building RESTful APIs**.

# @ExceptionHandler – What Does It Do?

@ExceptionHandler is used inside @ControllerAdvice (or directly inside a controller) to handle specific exceptions.
It tells Spring which method should be called when an exception occurs.

**EXAMPLE:**

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFoundException(UserNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<String> handleIllegalArgumentException(IllegalArgumentException ex) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("Invalid input: " + ex.getMessage());
    }
}
```

# Example: using inside the controller

```
@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping("/{id}")
    public User getUser(@PathVariable int id) {
        return userService.getUserById(id); // This may throw UserNotFoundException
    }

    // Exception handler inside the controller (No need for @ControllerAdvice)
    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFound(UserNotFoundException ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
    }
}
```

## Advantages of RESTful APIs in Spring Boot

✅ Fast Development – Spring Boot reduces the need for complex configurations, allowing developers to focus on writing code rather than setting up the environment. Built-in features like auto-configuration and embedded servers make development quicker and more efficient.

✅ Scalability – REST APIs in Spring Boot support horizontal scaling, meaning more servers can be added to handle increased traffic. This makes it easier to expand an application as user demand grows.

✅ Security – Spring Boot integrates security features like authentication (verifying user identity) and authorization (controlling user access). Tools like Spring Security help protect APIs from threats like unauthorized access and data breaches.

✅ **Microservices Ready** – REST APIs work well with microservices architecture, where applications are built as smaller, independent services that communicate over HTTP. This makes it easier to manage, update, and deploy different parts of an application separately.

✅ **Standardized Approach** – REST follows standard principles like using HTTP methods (GET, POST, PUT, DELETE) and structured responses (usually JSON). This consistency makes APIs easier to understand, use, and integrate with other applications.

# Spring Data JPA :

Spring Data JPA is a Spring-based framework that enables **repository support for JPA** by providing an abstraction layer over JPA. It makes it easier to implement **CRUD (Create, Read, Update, Delete) operations**, **query methods**, **paging and sorting**, and **custom queries** without writing boilerplate code.

**Key Features:**

✅ **Simplifies Data Access**
**Reduces the need for writing queries and boilerplate code.**
**Provides Repository interfaces that automatically generate implementations.**
✅ **Repository Abstraction**
**Offers interfaces like CrudRepository, JpaRepository, and PagingAndSortingRepository.**
✅ **Query Method Derivation**
**Allows you to define query methods in interfaces without needing to write SQL or JPQL.**
✅ **Paging and Sorting**
**Supports built-in pagination and sorting using Pageable and Sort.**
✅ **Custom Queries**
**Supports JPQL, Native Queries, and Criteria API.**

# Setting Up Spring Data JPA:

For a Spring Boot application, add the following dependencies to your pom.xml:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
</dependency>
```

**Configure application.properties:**

```
# Specifies the JDBC URL for connecting to the MySQL database.
# "localhost" means the database is running on the same machine as the application.
# "3306" is the default MySQL port.
# "mydb" is the name of the database.
spring.datasource.url=jdbc:mysql://localhost:3306/mydb

# The username and password  used to connect to the MySQL database.

spring.datasource.username=root
spring.datasource.password=root

# Specifies the Hibernate dialect for MySQL 8.
# This helps Hibernate generate optimized SQL statements for MySQL.

spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect

# Defines the Hibernate strategy for managing the database schema."update" means Hibernate will automatically update the schema without dropping existing data.
# Other options: "create" (drops & creates tables on startup), "create-drop" (drops tables on shutdown), "validate" (checks schema but doesn't modify).
spring.jpa.hibernate.ddl-auto=update
```

## Defining a JPA Entity in Spring Data JPA:

In Spring Data JPA, an entity represents a table in a relational database, and each instance of that entity corresponds to a row in the table. We use the @Entity annotation to mark a Java class as a JPA entity.

**Annotations:**

**@Entity :**

Marks a class as a JPA entity, meaning it maps to a database table.

**@Table(name = "users") :**

(Optional) Specifies the table name in the database. If not provided, it defaults to the class name.

**@Id :**

Declares the primary key for the entity.

**@GeneratedValue(strategy = GenerationType.IDENTITY) :**

Auto-generates primary key values (commonly used for MySQL).

**@Column(nullable = false, length = 50) :**

Maps a field to a database column with constraints (e.g., nullable = false ensures it's required).

**@Column(unique = true)** :

Ensures that the column values are unique.

## Creating a Repository in Spring Data JPA:

Spring Data JPA simplifies database interactions by providing **repository interfaces**, eliminating the need for boilerplate **DAO (Data Access Object) code**. With these repositories, you can easily perform CRUD (Create, Read, Update, Delete) operations without manually implementing them.

**What is a Repository?**
A **repository** in Spring Data JPA is an abstraction layer that allows interacting with a database without writing SQL queries manually. It provides built-in methods for performing CRUD operations and also allows defining custom query methods.
**EXAMPLE:**

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    // Custom method to find users by name (Spring Boot auto-generates the query)
    List<User> findByName(String name);
}
```

Spring Data JPA provides multiple repository interfaces that extend from **Spring's Repository hierarchy**:

**Repository<T, ID>** → A basic marker interface that serves as a parent for all repository interfaces. It does not provide any methods and should not be used directly.

**CrudRepository<T, ID>** → Extends Repository and provides basic CRUD (Create, Read, Update, Delete) operations, such as save(), findById(), delete(), etc.

**PagingAndSortingRepository<T, ID>** → Extends CrudRepository and adds support for pagination and sorting using methods like findAll(Pageable pageable) and findAll(Sort sort).

**JpaRepository<T, ID>** → Extends both CrudRepository and PagingAndSortingRepository, adding JPA-specific methods such as flush(), saveAndFlush(), and batch processing capabilities.

⚡ Recommended: JpaRepository<T, ID>
It provides the most features and is the preferred choice for JPA-based applications.

| Repository Type | CRUD (save(), findById(), delete()) | Pagination & Sorting | JPA-Specific (flush(), saveAndFlush(), batch processing) |
|---|---|---|---|
| **CrudRepository** | ✅ Yes | ❌ No | ❌ No |
| **PagingAndSortingRepository** | ✅ Yes | ✅ Yes | ❌ No |
| **JpaRepository** | ✅ Yes | ✅ Yes | ✅ Yes |

**EXAMPLE:**

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository // Marks this as a Spring Data JPA Repository
public interface UserRepository extends JpaRepository<User, Long> {

    // Custom Query Method: Find user by email
    Optional<User> findByEmail(String email);

    // Spring Data JPA will generate the implementation automatically
}
```

# JpaRepository Methods and Descriptions:

**save(User user):**

Saves a given entity. If the entity exists, it updates it; otherwise, it inserts a new record.

**saveAll(Iterable<User> users):**

Saves multiple entities in a batch operation.

**findById(Long id):**

Retrieves an entity by its primary key. Returns Optional<User> to handle null cases.

**findAll()** :

Retrieves all entities from the database table.

**findAll(Sort sort)** :

Retrieves all entities, sorted according to the given Sort parameter.

**findAllById(Iterable<Long> ids):**

Retrieves multiple entities by their IDs.

**deleteById(Long id):**

Deletes an entity by its primary key.

**delete(User user):**

Deletes a specific entity instance.

**deleteAll(Iterable<User> users):**

Deletes multiple entity instances.

**deleteAll()** :

Deletes all entities from the database table.

**existsById(Long id) :**

Checks whether an entity with the given ID exists in the database.

**count()** :

Returns the total number of entities in the table.

# Pagination:

Pagination is the process of fetching **data in small chunks (pages)** instead of retrieving all records at once.

**Example:** Instead of loading **1,000,000 users**, fetch **10,000 users at a time** using pages.

## 🔹 Why Use Pagination?

| Feature | Without Pagination 🚨 | With Pagination ✅ |
|---|---|---|
| **Performance** | Slow (large queries) | Fast (small queries) |
| **Memory Usage** | High (loads all data) | Low (fetches page-by-page) |
| **Response Time** | Slow API response | Faster API response |
| **Scalability** | Not scalable | Works for millions of records |

## How Pagination Works in Spring Data JPA?

Spring Data JPA provides the Pageable interface to handle pagination.

**Basic Syntax:**

```
Pageable pageable = PageRequest.of(pageNumber, pageSize);
Page<User> usersPage = userRepository.findAll(pageable);
```

**pageNumber** → Which page to fetch (starts from 0).
**pageSize** → How many records per page.

## 📌 Example:

PageRequest.of(1, 10) → Fetch Page 2, with 10 records per page.

**1. Pageable**

Pageable is an interface in Spring Data JPA used for pagination and sorting.
It defines the page number, page size, and sorting order.
The most common implementation of Pageable is PageRequest.

**Example:**

```
Pageable pageable = PageRequest.of(0, 10);
```

PageRequest.of(0, 10) creates a Pageable object:
0 → Page index (first page, as index starts from 0).
10 → Number of records per page.

**2. Page<T>**

Page<T> is a wrapper that contains a list of results (List<T>) along with pagination metadata.
**It includes useful details like:**
Total number of elements.
Total number of pages.
Current page number.
Number of elements in the current page.

**Example Usage in Repository:**

Assuming you have a User entity and a UserRepository that extends JpaRepository:
Page<User> userPage = userRepository.findAll(pageable);

Here:

findAll(pageable) fetches the first 10 users.
userPage contains both the user data and pagination details.

**Accessing Pagination Details:**

```
List<User> users = userPage.getContent();   // List of users in current page
int totalPages = userPage.getTotalPages();  // Total number of pages
long totalElements = userPage.getTotalElements(); // Total records in DB
boolean isFirst = userPage.isFirst();       // Check if it's the first page
boolean isLast = userPage.isLast();         // Check if it's the last page
```

**EXAMPLE:**

📌 Response Example (JSON Output)


```
{
  "content": [
    { "id": 6, "name": "Frank" },
    { "id": 7, "name": "Grace" },
    { "id": 8, "name": "Henry" },
    { "id": 9, "name": "Ivy" },
    { "id": 10, "name": "Jack" }
  ],
  "pageable": { "pageNumber": 1, "pageSize": 5 },
  "totalPages": 200000,
  "totalElements": 1000000,
  "last": false,
  "first": false
}
```

## 1️⃣ Default Pagination Using findAll()

```
Pageable pageable = PageRequest.of(0, 10); // Page 1, 10 records
Page<User> usersPage = userRepository.findAll(pageable);
List<User> users = usersPage.getContent(); // Get user list
```

## 2️⃣ Custom Query with Pagination

Use @Query with pagination for filtering data.

```
@Query("SELECT u FROM User u WHERE u.role = :role")
Page<User> findByRole(@Param("role") String role, Pageable pageable);
```

📌 Example Call:

```
Pageable pageable = PageRequest.of(0, 5, Sort.by("name").ascending());
Page<User> admins = userRepository.findByRole("ADMIN", pageable);
```

## 3️⃣ Sorting & Pagination Together:

Spring Data JPA supports pagination + sorting in a single query.

```
Pageable pageable = PageRequest.of(0, 5, Sort.by("name").ascending());
Page<User> usersPage = userRepository.findAll(pageable);
```

📌 Sorting Options:

```
Sort.by("name").ascending();    // Sort by name (A-Z)
Sort.by("age").descending();    // Sort by age (highest first)
Sort.by("city").ascending().and(Sort.by("name").descending()); // Multiple sorting
```

# Derived Query Methods:

Derived Query Methods are methods in **Spring Data JPA repositories** that allow you to **query the database without writing SQL**. Instead of writing complex JPQL or native SQL queries, you define **method names following a specific naming convention**, and Spring Data JPA automatically generates the required query for you.

### 📌 Where to Write Derived Query Methods?

You **define them inside a repository interface**, which interacts with the database using Spring Data JPA.

### 🔹 How Do Derived Query Methods Work?

You define a method in a repository interface, following a naming pattern.
Spring Data JPA interprets the method name and creates the SQL query automatically.
No need to write @Query or SQL manually.

- ✅ Saves Time
- ✅ Less Boilerplate Code
- ✅ Easy to Read & Maintain

- **1. Comparison Queries (Greater Than, Less Than, Between)**

| Operator | Method Naming Convention | Example Usage |
|---|---|---|
| Greater Than (>) | findByFieldNameGreaterThan() | findByAgeGreaterThan(30) |
| Greater Than or Equal (>=) | findByFieldNameGreaterThanEqual() | findBySalaryGreaterThanEqual(50000) |
| Less Than (<) | findByFieldNameLessThan() | findByPriceLessThan(1000) |
| Less Than or Equal (<=) | findByFieldNameLessThanEqual() | findByExperienceLessThanEqual(5) |
| Between (BETWEEN) | findByFieldNameBetween() | findBySalaryBetween(40000, 60000) |

**Example Code:**

```
List<Employee> findByAgeGreaterThan(int age);
List<Employee> findBySalaryLessThan(double salary);
List<Employee> findByJoinDateBetween(LocalDate start, LocalDate end);
```

- **2. Equality & Matching Queries**

| Operator | Method Naming Convention | Example Usage |
|---|---|---|
| Exact Match (=) | findByFieldName() | findByEmail("test@example.com") |
| Not Equal (!=) | findByFieldNameNot() | findByStatusNot("Inactive") |
| Ignore Case | findByFieldNameIgnoreCase() | findByNameIgnoreCase("john") |
| Containing (LIKE %value%) | findByFieldNameContaining() | findByDescriptionContaining("Spring") |
| Starting With (LIKE value%) | findByFieldNameStartingWith() | findByNameStartingWith("A") |
| Ending With (LIKE %value) | findByFieldNameEndingWith() | findByNameEndingWith("son") |

**Example Code:**

```
List<User> findByEmail(String email);
List<User> findByStatusNot(String status);
List<User> findByNameIgnoreCase(String name);
List<Product> findByDescriptionContaining(String keyword);
List<User> findByNameStartingWith(String prefix);
```

- **3. Logical Operators (AND, OR)**

| Operator | Method Naming Convention | Example Usage |
|---|---|---|
| **AND** | findByField1AndField2() | findByAgeGreaterThanAndSalaryLessThan(30, 50000) |
| **OR** | findByField1OrField2() | findByCityOrCountry("New York", "USA") |

**Example Code:**

```
List<Employee> findByAgeGreaterThanAndSalaryLessThan(int age, double salary);
List<Customer> findByCityOrCountry(String city, String country);
```

- **4. Date Queries**

| Operator | Method Naming Convention | Example Usage |
| --- | --- | --- |
| Before | findByDateBefore() | findByCreatedDateBefore(LocalDate.now()) |
| After | findByDateAfter() | findByJoinDateAfter(LocalDate.of(2020, 1, 1)) |

**Example Code:**

```
List<Order> findByCreatedDateBefore(LocalDate date);
List<Employee> findByJoinDateAfter(LocalDate date);
```

- **5. Sorting Queries**

| Operator | Method Naming Convention | Example Usage |
|---|---|---|
| Order By Ascending | findByFieldOrderByFieldAsc() | findByNameOrderBySalaryAsc() |
| Order By Descending | findByFieldOrderByFieldDesc() | findByAgeOrderBySalaryDesc() |

**Example Code:**

```
List<Employee> findByDepartmentOrderBySalaryAsc(String department);
List<User> findByCityOrderByNameDesc(String city);
```

## Key Benefits of Derived Query Methods

- No SQL Needed – Queries are automatically generated.
- Readability – Method names are self-explanatory.
- Less Code – No need for @Query annotations.
- Maintainability – Easier to modify queries.

# Custom Query Methods

Derived query methods are great, but they have limitations when queries become complex.
If you need more control, use Custom Query Methods with @Query.

**Using @Query for Custom Queries:**

The @Query annotation allows you to define custom JPQL (Java Persistence Query Language) or native SQL queries in a Spring Data JPA repository. This is useful when built-in JPA methods (findById, findAll, etc.) are not sufficient for complex queries.

**Types of Queries in @Query:**

You can write different types of queries using @Query in Spring Data JPA:

**1. JPQL Queries (Java Persistence Query Language)**

JPQL is similar to SQL but operates on **entity objects** instead of database tables.

**Characteristics of JPQL**

- Works with **entity classes** and their attributes.
- Queries are independent of the underlying database (DBMS).
- Uses **Java class names** and **field names** instead of table names and column names.
- Can perform CRUD operations like SELECT, INSERT, UPDATE, and DELETE.

**Example:**

```
@Query("SELECT u FROM User u WHERE u.name = :name")
List<User> searchByName(@Param("name") String name);
```

This searches for users based on their name field.
It operates on User entity (u refers to User).
:name is a named parameter, passed dynamically.

When writing JPQL (Java Persistence Query Language) or Native SQL Queries in Spring Data JPA, we use parameters to avoid hardcoding values and to prevent SQL injection.

## There are two types of parameters:

## 1 Positional Parameters (?1, ?2, etc.):

- Uses ? followed by a number (?1, ?2, etc.).
- The number represents the position of the argument in the method.
- Works well for simple queries but can be confusing with many parameters.

EXAMPLE:

```
@Query("SELECT u FROM User u WHERE u.id = ?1 AND u.name = ?2")
User findUserByIdAndName(Long id, String name);
```

✓ **How it works?**

?1 → Replaced by the first method parameter (id).

- ?2 → Replaced by the second method parameter (name).

## 2 Named Parameters (:name):

Uses :parameterName instead of ?1, ?2, etc.

- More readable and clearer than positional parameters.
- Preferred for queries with multiple parameters.

Example:

```
@Query("SELECT u FROM User u WHERE u.email = :email AND u.status = :status")
User findUserByEmailAndStatus(@Param("email") String email, @Param("status") String status);
```

✓ **How it works?**

:email → Replaced by the email parameter.
:status → Replaced by the status parameter.

## 2. Native SQL Queries

A **native query** in JPA allows developers to use raw SQL statements to interact with the database directly. Unlike JPQL, native queries are **database-dependent**.

### Characteristics of Native Query

- Uses raw SQL queries.
- Can use database-specific functions.
- Can be more efficient for complex queries.
- Cannot leverage JPA's object-relational mapping directly.

**Example:**

```
@Query(value = "SELECT * FROM users WHERE name = :name", nativeQuery = true)
List<User> findByNameNative(@Param("name") String name);
```

Here, users refers to the actual database table, not the entity.

## 3. Named Query:

A Named Query is a predefined query that is declared in the entity class using @NamedQuery annotation. Named queries can be JPQL or native queries.

## Characteristics of Named Query

- Improves code **readability** and **maintainability**.
- Predefined and can be **reused** multiple times.
- Optimized at startup for better **performance**.
- Supports both **JPQL** and **Native Queries**.

**Example:**

```
import jakarta.persistence.*;

@Entity
@Table(name = "employees")
@NamedQuery(name = "Employee.findByDepartment", query = "SELECT e FROM Employee e WHERE
e.department = :dept")
@NamedNativeQuery(name = "Employee.findBySalary", query = "SELECT * FROM employees WHERE salary >
:salary", resultClass = Employee.class)
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String department;
    private double salary;

    // Getters and Setters
}
```

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Using Named JPQL Query
    @Query(name = "Employee.findByDepartment")
    List<Employee> findByDepartment(@Param("dept") String department);

    // Using Named Native Query
    @Query(name = "Employee.findBySalary")
    List<Employee> findBySalary(@Param("salary") double salary);
}
```

## Which One Should You Use?

- **Use JPQL** if your query is simple and should be **database-independent**.
- **Use Native Query** if you need **database-specific optimizations**.
- **Use Named Queries** if you need to **reuse** the same query multiple times and want **better performance**.

# Thank you